CS F469: Information Retrieval Assignment 1

Anmalchara Amruta Murgunde (2021A7PS0205H)

Disclaimer: To build a search engine for a given corpus of documents.

```
# imports
from glob
from query_functions import search
from setup import startup_engine
import os

# path for our data (going one directory above)
workir = os.getcwd()
data_path = os.path.join(workir,"data")
inv_path = os.path.join(workir,"invdata/invindex"
import glob
corp = glob.glob(inv_path + "*.txt")
```

Step 1: Pipeline for extracting, cleaning and tokenizing text from pdfs

1.1: Extracting text from pdfs

The pdfs are extracted using the `fitz` Python library. `fitz` is a wrapper around the `PyMuPDF` library. A pipeline is created to extract text from the pdfs and save them as `.jsl` files. The pipeline is as follows:

1. Extract text from pdfs using `fitz`
2. Store the contents of each file in a dictionary with `"text"` as the key and the extracted text separated page-wise in a list as the value. (in a `.jsl` file)
3. Repeat for all pdfs and separate them in the same folder structure as the original pdfs.

Note: Implementation of this step can be found in the `cleaning.py` file. (fully documented)

1.2: Lemmatizing and creation of posting lists

From the created `.jsl` files we create a `DataFrame` with the following columns:

1. `id`: to uniquely identify passages (Document for our engine)
2. `text`: the text of the passage (split based on two newlines `\n\n` and `\r\n` and restricted to ~ 3000 characters per passage)
3. `tokenized`: the lemmatized version of the text in the passage(s) (`spaCy` was used for lemmatization)
4. `posting_list`: the posting list for the passage (unique lemmas from `tokenized`)
5. `document_name`: the name of the pdf file from which the passage was extracted
6. `page_number`: the page number of the pdf from which the passage was extracted
7. `paragraph_number`: the paragraph number of the page from which the passage was extracted

Note: Implementation of the above steps can be found in the `tokenizing.py` file. (fully documented)

Note 2: The entire process outlined till here is in a pipeline, so adding new pdfs, new folders will not need any change of code except for the paths in the `cleaning.py` file.

Step 2: Construction of inverted index and permuterm indexes

2.1: Inverted index

The inverted index which is used for boolean querying and ranking later on is constructed using the `posting_list` column of the `DataFrame` created in the previous step. The inverted index is a dictionary with the keys as the unique lemmas and the values as the sorted list of documents in which the lemma occurs.

The documents are sorted based on the `id` column of the `DataFrame` and the `id` is the index of the document in the `DataFrame`.

The data structure used for this is the default Python `dict`, whose keys are `str`s and values are `LinkedList`s (implemented in `LinkedList.py`)

Visual example:

```
{
    "lemma1": [12] -> [34] -> [56] -> [78] -> [90],
    "lemma2": [12] -> [2] -> [3] -> [4] -> [5] -> [6] -> [7] -> [8] -> [9] -> [10],
}
```

2.2: Permuterm indexes

The permuterm indexes (normal and reverse) are constructed using the `inverted_index` created in the previous step. The permuterm indexes are dictionaries with the keys as the permuterm and the values as the sorted list of documents in which the permuterm occurs.

Like the inverted index, the documents are sorted based on the `id` column of the `DataFrame` and the `id` is the index of the document in the `DataFrame`.

Visual example:

1. Normal permuterm index

```
{
    "lemma$": [12] -> [34] -> [56] -> [78] -> [90],
    "lemma$": [12] -> [2] -> [3] -> [4] -> [5] -> [6] -> [7] -> [8] -> [9] -> [10],
}
```

1. Reverse permuterm index

```
{
    "$lemma": [12] -> [34] -> [56] -> [78] -> [90],
    "$lemma": [12] -> [2] -> [3] -> [4] -> [5] -> [6] -> [7] -> [8] -> [9] -> [10],
}
```

2.3 Biword index

The biword index is used to query `phrase` queries. It is constructed using the `tokenized` column of the `DataFrame` created in the previous step. The biword index is a dictionary with the keys as the unique biwords and the values as the sorted list of documents in which the biword occurs.

Visual example:

```
{
    "lemma1 lemma2": [12] -> [34] -> [56] -> [78] -> [90],
    "lemma2 lemma3": [12] -> [2] -> [3] -> [4] -> [5] -> [6] -> [7] -> [8] -> [9] -> [10],
}
```

Implementation of the above steps can be found in the `setup.py` file. (fully documented)

Note: The indexes could not be saved as `.pkl` files due to the size of the indexes (hitting the maximum possible recursion limit in Python). So instead of saving these indexes, there is currently a function `start_engine` which creates and returns these indexes along with the main `DataFrame` which is used for lookups.

```
inverted_list, perm_index, rev_perm_index, n_word_index, _, main_df = startup_engine("invx)
```

Important statistics

While the `start_engine` function takes around `30s`, to create all required indexes, this is only a 1 time run per startup, and any subsequent query will be answered in `~0.1s` (for boolean retrieval and ranking) as can be seen later on.

Step 3: Boolean retrieval/filtering

The boolean filter can either be used:

1. Separately, if the user only wishes to filter the documents based on the query
2. As a precursor to the ranking algorithm, if the user wishes to rank the documents based on the query, so that we want have to rank documents which do not match the query.

3.1: Free queries

Free implemented queries are:

1. `OR` queries: by default all words passed in the query string are treated as `OR` queries. For example, the query `information retrieval` will return all documents which contain either `information` or `retrieval`, or both.
2. `AND` queries: if any words in the query string are enclosed in double quotes, they are treated as `AND` queries. For example, the query `"information retrieval"` will return all documents which contain both `information` and `retrieval`. All other words not enclosed in double quotes (the `OR` parts of the query) won't be considered in the filtering. The ranking algorithm however will consider all words in the query.

```
def user_search(query, ranked=True, is_phrase=False, summarize=False, num_docs=None, spell_check=False, autocomplete=False, k_auto_results=5):
    '''Simular function to call the engine's search function without passing in the indexes
    Returns:
        return search(query, inverted_list, perm_index, rev_perm_index, n_word_index, main_df, ranked=ranked, is_phrase=is_phrase, show_summary=summarize, retrieve_n = num_docs, spell_check=spell_check, autocomplete=autocomplete

# defaults to a ranked 'OR' query
user_search("protection officer", num_docs=5)

Documents Retrieved: 1
..........
..........
Rank: 1
Document Name: complete-property-owner-policy-writing-policies-incepting-or-renewing-from-01042019-ac000b-11
Page Number: 63
Paragraph Number: 8
Score: 21.6541129782756500
..........
..........
Paragraph Text:
Bibhaphale Property Owner Policy WordingHome your rightness individual whose personal information we hold has theright to:• object to us processing it. We will either agree to stopprocessing or explain why we are unable to (the right toobject) ask for a copy of their personal information we hold, subjectto certain exemptions (a data subject access request)• ask us to update or correct their personal information inexact or inaccurate (the right tocorrection) ask us to delete their personal information fromour record(s) if it is no longer needed for the original purpose (the right tobe forgotten)• ask us to restrict the processing of their personal information incertain circumstances (the right of restriction)• ask for a copy of their personal information, so it can be usedfor their own purposes (the right to data portability)• complain if they feel their personal information has beenmishandled. We encourage individuals to come to us in thefirst instance but they are entitled to complain directly to theInformation Commissioner's Office (ICO) www.ico.org.uk• ask us, at any time, to stop processing the personal information, if the processing is based only on individualconsent (the right to withdraw consent)If you wish to exercise any of these rights please contact ourCustomer Satisfaction Manager Address: Customer Satisfaction Manager, Allianz, 57 Lakewood,Guildford, Surrey, GU1 1DBEmail: accidentadjuster.in.ukPhone: 01483 5324918 Allianz (UK) Group Data Protection Officer Contact details Allianz Insurance plc and Allianz Engineering Inspection Services Limited are companies within the Allianz Holdings Any queries about how we use personal information should beaddressed to our Data Protection Officer Address: Data Protection Officer, Allianz, 57 Lakewood, Guildford Surrey, GU1 1DBEmail: dataprotectionofficer@allianz.co.ukPhone: 0330 102 1837Underlying personal information may share personal information with:• other companies within the Allianz Holdings Group,• other companies within the Allianz Group,• other
```